ANALOG DEVICES

ADI TFLITE MICRO SHARCFX LIBRARY PRODUCT REFERENCE GUIDE

ANALOG DEVICES, INC.

www.analog.com

(3.0.0, July 2025)

Table of Contents

Copyright, Disclaimer & Trademark Statements

Copyright Information

Disclaimer

Analog Devices, Inc. ("Analog Devices") reserves the right to change this product without prior notice. Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use; nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under the patent or other rights of Analog Devices

Trademark and Service Mark Notice

Analog Devices, the Analog Devices logo, Blackfin, SHARC, TigerSHARC, CrossCore, VisualDSP, VisualDSP++, EZ-KIT Lite, EZ-Extender, SigmaStudio and Collaborative are the exclusive trademarks and/or registered trademarks of Analog Devices, Inc ("Analog Devices").

All other brand and product names are trademarks or service marks of their respective owners.

Analog Devices' Trademarks and Service Marks may not be used without the express written consent of Analog Devices, such consent only to be provided in a separate written agreement signed by Analog Devices. Subject to the foregoing, such Trademarks and Service Marks must be used according to Analog Devices' Trademark Usage guidelines. Any licensee wishing to use Analog Devices' Trademarks and Service Marks must obtain and follow these guidelines for the specific marks at issue.

1 Introduction

This document describes the API of TFLM SharcFX Optimized routines.

1.1 Scope

The document is intended to assist software developers using these library modules to their application.

1.2 Organization of this Guide

Section 1 : this section contains the introduction.

Section 2 : contains the programmer's reference.

1.3 Acronyms

ADI	Analog Devices Inc.
API	Application Program Interface

1.4 Additional Information

1.4.1 Library Evaluation Version

There is no crippling enabled in the version of the library provided with this release.

1.4.2 Other Information

For more information on the latest ADI processors, silicon errata, code examples, development tools, system services and devices drivers, technical support and any other additional information, please visit our website at www.analog.com/processors.

2 Programmer's Reference

The ADI optimized SharcFX API Library is organized as follows :

- Section 2.1 Files
- Section 2.2 Functions

2.1 Files

2.1.1 Include Files

The adi_sharcfx_nn library contains the optimized routines for TFLM under the following path ***adi_sharcfx_nn\Project\src***

- ***adi_sharcfx_nn.h***
- ***adi_sharcfx_common.h***

2.1.2 Source Files

The optimized routines Library consists of following source files in the folder ***adi_sharcfx_nn \Project\src***

- ***adi_sharcfx_activations.cpp***
- ***adi_sharcfx_conv2d.cpp***
- ***adi_sharcfx_depthconv2d.cpp***
- ***adi_sharcfx_elementwise_ops.cpp***
- ***adi_sharcfx_fully_connected.cpp***
- ***adi_sharcfx_maxpool.c***

2.2 Functions

This section contains a detailed description of the optimized functions.

2.2.1 Convolution 2d

There are several variants of the 2D convolution operation designed to handle different cases. The specific convolution function used depends on the nature of the input.

2.2.1.1 adi_sharcfx_conv2d_dilation1x1_int8

Syntax

```
void adi_sharcfx_conv2d_dilation1x1_int8 (
      const int8_t* pInputBuffer,
      const int8_t* pWeightsBuffer,
      const int32_t* pBiasBuffer,
      int8_t* pOutputBuffer,
      int32_t nBatches,
      int32_t nInChannels,
      int32_t nOutChannels,
      int32_t nKernelHeight,
      int32_t nKernelWidth,
      int32_t nNumKernels,
      int32_t nInputWidth,
      int32_t nInputHeight,
      int32_t stride_height,
      int32_t stride_width,
      int32_t nPadHeight,
      int32_t nPadWidth,
      int32_t nOutHeight,
      int32_t nOutWidth,
      int32_t *pQuantizedMultiplier,
      int32_t *pQuantizedShift,
      int32_t pInZeroPoint,
      int32_t pOutZeroPoint,
      int32_t nFilterZeroPoint,
      int32_t nActMin,
      int32_t nActMax
);
```

Description

This function is used to do the Conv2d operation with any m x n filter, 8-bit interleaved input given the dilation factor for the convolution kernel is (1,1).

Parameters

Name: pInputBuffer

Type: const int8_t*

Direction: Input
Description: Pointer to the input buffer.

Name: pWeightsBuffer
Type: const int8_t*
Direction: Input
Description: Pointer to the input weights buffer.

Name: pBiasBuffer
Type: const int32_t*
Direction: Input
Description: Pointer to the input bias buffer.

Name: pOutputBuffer_
Type: nt8_t*
Direction: Output
Description: Pointer to the output buffer.

Name: nBatches
Type: int32_t
Direction: Input
Description: Input param which is the number of batches.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of inputs channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of outputs channels.

Name: nKernelHeight
Type: int32_t
Direction: Input
Description: Input param which is the size of the kernel height.

Name: nKernelWidth
Type: int32_t
Direction: Input
Description: Input param which is the size of the kernel width.

Name: nNumKernels
Type: int32_t
Direction: Input
Description: Input param which is the number of kernels.

Name: nInputWidth
Type: int32_t
Direction: Input
Description: Input param which is the input width size.

Name: nInputHeight

Type: int32_t
Direction: Input
Description: Input param which is the input height size.

Name: stride_height
Type: int32_t
Direction: Input
Description: Input param which is the kernel's height size.

Name: stride_width
Type: int32_t
Direction: Input
Description: Input param which is the kernel's width size.

Name: nPadHeight
Type: int32_t
Direction: Input
Description: Input param which is the padding's height size.

Name: nPadWidth
Type: int32_t
Direction: Input
Description: Input param which is the padding's width size.

Name: nOutHeight
Type: int32_t
Direction: Input

Description: Input param which is the Output's height size.

Name: nOutWidth
Type: int32_t
Direction: Input
Description: Input param which is the Output's width size.

Name: pQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input Quantized Multiplier buffer.

Name: pQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input Quantized Shift buffer.

Name: pInZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the Input's zero point.

Name: pOutZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the Output's zero point.

Name: nFilterZeroPoint

Type: int32_t

Direction: Input

Description: Input param which is the Filter's zero point.

Name: nActMin

Type: int32_t

Direction: Input

Description: Input param which is the activation function minimum value.

Name: nActMax

Type: int32_t

Direction: Input

Description: Input param which is the activation function maximum value.

Return value.

None.

2.2.1.2 adi_sharcfx_conv2d_kernel3x3_stride1_valid_pad_int8

```
void adi_sharcfx_conv2d_kernel3x3_stride1_valid_pad_int8 (
            const int8_t* pInputBuffer,
            const int8_t* pWeightsBuffer,
            const int32_t* pBiasBuffer,
            int8_t* pOutputBuffer,
            int32_t nInChannels,
            int32_t nOutChannels,
            int32_t nWidth,
            int32_t nHeight,
            int32_t nFilters,
            int32_t *nQuantizedMultiplier,
            int32_t *nQuantizedShift,
            int32_t pInZeroPoint,
            int32_t pOutZeroPoint
);
```

Description

This function is used to do the Conv2d operation with 3x3 filter, 8-bit input, stride 1x1 and valid padding.

Parameters

Name: pInputBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input buffer.

Name: pWeightsBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer

Type: const int32_t*

Direction: Input

Description: Pointer to the input bias buffer.

Name: pOutputBuffer
Type: int8_t *
Direction: Output
Description: Pointer to the output buffer.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of input channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of output channels.

Name: nWidth
Type: int32_t
Direction: Input
Description: Input param which is the filter's width.

Name: nHeight
Type: int32_t
Direction: Input
Description: Input param which is the filter's height.

Name: nFilters
Type: int32_t
Direction: Input
Description: Input param which is the number of filters.

Name: nQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: nQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized shift buffer.

Name: pInZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the input zero point.

Name: pOutZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the output zero point.

Return value.

None.

2.2.1.3 adi_sharcfx_conv2d_kernel3x3_stride1_same_pad_int8

```
void adi_sharcfx_conv2d_kernel3x3_stride1_same_pad_int8 (
      const int8_t* pInputBuffer,
      const int8_t* pWeightsBuffer,
      const int32_t* pBiasBuffer,
      int8_t* pOutputBuffer,
      int32_t nInChannels,
      int32_t nOutChannels,
      int32_t nWidth,
      int32_t nHeight,
      int32_t nFilters,
      int32_t *nQuantizedMultiplier,
      int32_t *nQuantizedShift,
      int32_t pInZeroPoint,
      int32_t pOutZeroPoint
);
```

Description

This function is used to do the Conv2d operation with 3x3 filter, 8-bit input, stride 1x1 and same padding.

Parameters

Name: pInputBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input buffer.

Name: pWeightsBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer

Type: const int32_t*

Direction: Input

Description: Pointer to the input bias buffer.

Name: pOutputBuffer
Type: int8_t *
Direction: Output
Description: Pointer to the output buffer.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of input channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of output channels.

Name: nWidth
Type: int32_t
Direction: Input
Description: Input param which is the filter's width.

Name: nHeight
Type: int32_t
Direction: Input
Description: Input param which is the filter's height.

Name: nFilters
Type: int32_t
Direction: Input
Description: Input param which is the number of filters.

Name: nQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: nQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized shift buffer.

Name: pInZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the input zero point.

Name: pOutZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the output zero point.

Return value.

None.

2.2.1.4 adi_sharcfx_conv2d_kernel3x3_stride2_valid_pad_int8

```
void adi_sharcfx_conv2d_kernel3x3_stride2_valid_pad_int8 (
            const int8_t* pInputBuffer,
            const int8_t* pWeightsBuffer,
            const int32_t* pBiasBuffer,
            int8_t* pOutputBuffer,
            int32_t nInChannels,
            int32_t nOutChannels,
            int32_t nWidth,
            int32_t nHeight,
            int32_t nFilters,
            int32_t *nQuantizedMultiplier,
            int32_t *nQuantizedShift,
            int32_t pInZeroPoint,
            int32_t pOutZeroPoint
);
```

Description

This function is used to do Conv2d operation with 3x3 filter, 8-bit input, stride 2x2 and valid padding.

Parameters

Name: pInputBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input buffer.

Name: pWeightsBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer

Type: const int32_t*

Direction: Input

Description: Pointer to the input bias buffer.

Name: pOutputBuffer
Type: int8_t *
Direction: Output
Description: Pointer to the output buffer.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of input channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of output channels.

Name: nWidth
Type: int32_t
Direction: Input
Description: Input param which is the filter's width.

Name: nHeight
Type: int32_t
Direction: Input
Description: Input param which is the filter's height.

Name: nFilters
Type: int32_t
Direction: Input
Description: Input param which is the number of filters.

Name: nQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: nQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized shift buffer.

Name: pInZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the input zero point.

Name: pOutZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the output zero point.

Return value.

None.

2.2.2 Pointwise convolution

2.2.2.1 adi_sharcfx_conv2d_kernel1x1_int8

```
void adi_sharcfx_conv2d_kernel1x1_int8 (
            const int8_t* pInputBuffer,
            const int8_t* pWeightsBuffer,
            const int32_t* pBiasBuffer,
            int8_t* pOutputBuffer,
            int32_t nBatches,
            int32_t nInChannels,
            int32_t nOutChannels,
            int32_t nSize,
            int32_t *nQuantizedMultiplier,
            int32_t *nQuantizedShift,
            int32_t nInputOffset,
            int32_t nOutputOffset
);
```

Description

This function is used to do the Conv2d operation with 1x1 filter, 8-bit input, and expects interleaved data.

Parameters

Name: pInputBuffer

Type: const int8_t*

Direction: Input

Description: Pointer to the input Input buffer.

Name: pWeightsBuffer

Type: const int8_t*

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer

Type: const int32_t *

Direction: Input

Description: Pointer to the input bias buffer.

Name: pOutputBuffer

Type: int8_t*

Direction: Output

Description: Pointer to the output buffer.

Name: nBatches

Type: int32_t

Direction: Input

Description: Input param which is the number of batches.

Name: nInChannels

Type: int32_t

Direction: Input

Description: Input param which is the number of input channels.

Name: nOutChannels

Type: int32_t

Direction: Input

Description: Input param which is the number of output channels.

Name: nSize

Type: int32_t

Direction: Input

Description: Input param which is the size of the input buffer.

Name: nQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: nQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized shift buffer.

Name: nInputOffset
Type: int32_t
Direction: Input
Description: Input param which is the Input offset value.

Name: nOutputOffset
Type: int32_t
Direction: Output
Description: Output param which is the output offset value.

Return value.

None.

2.2.2.2 adi_sharcfx_conv2d_kernel1x1_noninterleaved_int16

```
void adi_sharcfx_conv2d_kernel1x1_noninterleaved_int16 (
            const int16_t *pInputBuffer,
            int16_t *pOutputBuffer,
            const int8_t *pWeightsBuffer,
            const int32_t *pBias,
            int32_t  nOutWidth,
            int32_t  nInChannels,
            int32_t  nOutChannels,
            uint32_t *pQuantizedMultiplier,
            int32_t *pQuantizedShift,
            int32_t pInZeroPoint,
            int32_t pOutZeroPoint
);
```

Description

This function is used to do the Conv2d with 1x1 filter, 16-bit input, and expects non-interleaved data.

Parameters

Name: pInputBuffer

Type: const int16_t *

Direction: Input

Description: Pointer to the input buffer.

Name: pOutputBuffer

Type: int16_t *

Direction: Output

Description: Pointer to the output buffer.

Name: pWeightsBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer
Type: const int32_t *
Direction: Input
Description: Pointer to the input bias buffer.

Name: nOutWidth
Type: int32_t
Direction: Input
Description: Input param which is the output width.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of input channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of output channels.

Name: nQuantizedMultiplier
Type: uint32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: nQuantizedShift

Type: int32_t *

Direction: Input

Description: Pointer to the input quantized shift buffer.

Name: pInZeroPoint

Type: int32_t

Direction: Input

Description: Input param which is the input zero point.

Name: pOutZeroPoint

Type: int32_t

Direction: Input

Description: Input param which is the output zero point.

Return value.

None

2.2.3 Depth-wise convolution

2.2.3.1 adi_sharcfx_depthconv2d_int8

```
void adi_sharcfx_depthconv2d_int8 (
            const int8_t *pInputBuffer,
            int8_t *pOutputBuffer,
            const int8_t *pWeightsBuffer,
            const int32_t *pBiasBuffer,
            int32_t nInputWidth,
            int32_t nInputHeight,
            int32_t nDepthMult,
            int32_t nInChannels,
            int32_t nOutChannels,
            int32_t nKernelSizeWidth,
            int32_t nKernelSizeHeight,
            int32_t nTotalPaddingWidth,
            int32_t nTotalPaddingHeight,
            int32_t *pQuantizedMultiplier,
            int32_t *pQuantizedShift,
            int32_t pInZeroPoint,
            int32_t pOutZeroPoint,
            int32_t nStrideWidth,
            int32_t nStrideHeight,
            int32_t nActMin,
            int32_t nActMax

);
```

Description

This function is used to do the depth wise 2D convolution operation.

Parameters

Name: pInputBuffer

Type: const int8_t *

Direction: Input

Description: Pointer to the input buffer.

Name: pOutputBuffer

Type: int8_t *

Direction: Output
Description: Pointer to the output buffer.

Name: pWeightsBuffer
Type: const int8_t *
Direction: Input
Description: Pointer to the input weights buffer.

Name: pBiasBuffer
Type: const int32_t *
Direction: Input
Description: Pointer to the input bias buffer.

Name: nInputWidth
Type: int32_t
Direction: Input
Description: Input param which is the Input's width size.

Name: nInputHeight
Type: int32_t
Direction: Input
Description: Input param which is the Input's height size.

Name: nDepthMult
Type: int32_t
Direction: Input
Description: Input param which is the Input's depth multiplier.

Name: nInChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of input channels.

Name: nOutChannels
Type: int32_t
Direction: Input
Description: Input param which is the number of output channels.

Name: nKernelSizeWidth
Type: int32_t
Direction: Input
Description: Input param which is the filter's width.

Name: nKernelSizeHeight
Type: int32_t
Direction: Input
Description: Input param which is the filter's height.

Name: nTotalPaddingWidth
Type: int32_t
Direction: Input
Description: Input param which is the width of the padding.

Name: nTotalPaddingHeight

Type: int32_t
Direction: Input
Description: Input param which is the height of the padding.

Name: pQuantizedMultiplier
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized multiplier buffer.

Name: pQuantizedShift
Type: int32_t *
Direction: Input
Description: Pointer to the input quantized shift buffer.

Name: pInZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the input zero point.

Name: pOutZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the output zero point.

Name: nStrideWidth
Type: int32_t
Direction: Input

Description: Input param which is the stride's width.

Name: nStrideHeight
Type: int32_t
Direction: Input
Description: Input param which is the stride's height.

Name: nActMin
Type: int32_t
Direction: Input
Description: Input param which is the activation function minimum value.

Name: nActMax
Type: int32_t
Direction: Input
Description: Input param which is the activation function maximum value.

Return value.

None.

2.2.4 Pooling functions

2.2.4.1 adi_sharcfx_maxpool_int8

```
void adi_sharcfx_maxpool_int8(
            const int32_t input_y,
            const int32_t input_x,
            const int32_t output_y,
            const int32_t output_x,
            const int32_t stride_y,
            const int32_t stride_x,
            const int32_t kernel_y,
            const int32_t kernel_x,
            const int32_t pad_y,
            const int32_t pad_x,
            const int32_t act_min,
            const int32_t act_max,
            const int32_t ch_src,
            const int8_t *src,
            int8_t *dst
);
```

Description

This function is used to do the max pooling operation for 8-bit data.

Parameters

Name: input_y

Type: const int32_t

Direction: Input

Description: Input param which is the Input's height size.

Name: input_x

Type: const int32_t

Direction: Input

Description: Input param which is the Input's width size.

Name: output_y_

Type: const int32_t

Direction: Input

Description: Input param which is the Output's height size.

Name: output_x
Type: const int32_t
Direction: Input
Description: Input param which is the Output's width size.

Name: stride_y
Type: const int32_t
Direction: Input
Description: Input param which is the stride's height.

Name: stride_x
Type: const int32_t
Direction: Input
Description: Input param which is the stride's width.

Name: kernel_y
Type: const int32_t
Direction: Input
Description: Input param which is the filter's height.

Name: kernel_x
Type: const int32_t
Direction: Input
Description: Input param which is the filter's width.

Name: pad_y
Type: const int32_t
Direction: Input
Description: Input param which is the padding's height

Name: pad_x
Type: const int32_t
Direction: Input
Description: Input param which is the padding's width.

Name: act_min
Type: const int32_t
Direction: Input
Description: Input param which is the activation's minimum value.

Name: act_max
Type: const int32_t
Direction: Input
Description: Input param which is the activation's maximum value.

Name: ch_src
Type: const int32_t
Direction: Input
Description: Input param which is the number of channels of the input.

Name: src
Type: const int8_t *

Direction: Input

Description: Pointer to the input buffer.

Name: dst

Type: int8_t *

Direction: Output

Description: Pointer to the output buffer.

Return value.

None.

2.2.5 Fully connected layer

2.2.5.1 adi_sharcfx_fully_connected_int16

```
void adi_sharcfx_fully_connected_int16 (
      const int16_t* pInputBuffer,
      const int8_t* pWeightsBuffer,
      const int64_t* pBiasBuffer,
      int16_t* pOutputBuffer,
      int32_t nFilterDepth,
      int32_t nOutsize,
      int32_t nBatches,
      uint32_t nQuantizedMultiplier,
      int32_t nQuantizedShift,
      int32_t nInputOffset,
      int32_t nFilterOffset,
      int32_t nOutputOffset,
      int32_t output_activation_min,
      int32_t output_activation_max
);
```

Description

This function is used to do the fully connected layer operation with 16-bit input.

Name: pInputBuffer

Type: const int16_t*
Direction: Input
Description: Pointer to the input buffer.

Name: pWeightsBuffer
Type: const int8_t*
Direction: Input
Description: Pointer to the input weights buffer.

Name: pBiasBuffer
Type: const int64_t*
Direction: Input
Description: Pointer to the input bias buffer.

Name: pOutputBuffer
Type: int16_t*
Direction: Output
Description: Pointer to the output buffer.

Name: nFilterDepth
Type: int32_t
Direction: Input
Description: Input param which is the filter's depth.

Name: nOutsize
Type: int32_t
Direction: Input

Description: Input param which is the output size.

Name: nBatches
Type: int32_t
Direction: Input
Description: Input param which is the number of batches.

Name: nQuantizedMultiplier
Type: uint32_t
Direction: Input
Description: Input param which is the quantized multiplier value.

Name: nQuantizedShift
Type: int32_t
Direction: Input
Description: Input param which is the quantized shift value.

Name: nInputOffset
Type: int32_t
Direction: Input
Description: Input param which is the Input offset value

Name: nFilterOffset
Type: int32_t
Direction: Input
Description: Input param which is the filter offset value

Name: nOutputOffset

Type: int32_t

Direction: Input

Description: Input param which is the Output offset value

Name: output_activation_min

Type: int32_t

Direction: Input

Description: Input param which is the activation function minimum value.

Name: output_activation_max

Type: int32_t

Direction: Input

Description: Input param which is the activation function maximum value.

Return value.

None.

2.2.5.2 adi_sharcfx_fully_connected_int8

```
void adi_sharcfx_fully_connected_int8(
      const int8_t* pInputBuffer,
      const int8_t* pWeightsBuffer,
      const int32_t* pBiasBuffer,
      int8_t* pOutputBuffer,
      int32_t nFilterDepth,
      int32_t nOutsize,
      int32_t nBatches,
      uint32_t nQuantizedMultiplier,
      int32_t nQuantizedShift,
      int32_t nInputOffset,
      int32_t nFilterOffset,
      int32_t nOutputOffset,
      int32_t output_activation_min,
      int32_t output_activation_max
);
```

Description

This function is used to do the fully connected layer operation with 8-bit input.

Name: pInputBuffer

Type: const int8_t*

Direction: Input

Description: Pointer to the input buffer.

Name: pWeightsBuffer

Type: const int8_t*

Direction: Input

Description: Pointer to the input weights buffer.

Name: pBiasBuffer

Type: const int32_t*

Direction: Input

Description: Pointer to the bias buffer.

Name: pOutputBuffer
Type: int8_t*
Direction: Output
Description: Pointer to the output buffer.

Name: nFilterDepth
Type: int32_t
Direction: Input
Description: Input param which is the filter's depth.

Name: nOutsize
Type: int32_t
Direction: Input
Description: Input param which is the output size.

Name: nBatches
Type: int32_t
Direction: Input
Description: Input param which is the number of batches.

Name: nQuantizedMultiplier
Type: uint32_t
Direction: Input
Description: Input param which is the quantized multiplier value.

Name: nQuantizedShift
Type: int32_t
Direction: Input
Description: Input param which is the quantized shift value.

Name: nInputOffset
Type: int32_t
Direction: Input
Description: Input param which is the Input offset value

Name: nFilterOffset
Type: int32_t
Direction: Input
Description: Input param which is the filter offset value

Name: nOutputOffset
Type: int32_t
Direction: Input
Description: Input param which is the Output offset value

Name: output_activation_min
Type: int32_t
Direction: Input
Description: Input param which is the activation's minimum value.

Name: output_activation_max
Type: int32_t

Direction: Input

Description: Input param which is the activation's maximum value.

Return value.

None.

2.2.6 Activation functions

2.2.6.1 adi_sharcfx_logistic_int16

```
void adi_sharcfx_logistic_int16 (
            int32_t nInputMultiplier,
            int32_t nInputLeftShift,
            int32_t nInputSize,
            const int16_t* pInputData,
            int16_t* pOutputData
);
```

NOTE:

This version of the optimized logistic function has a tolerance of +/- 106[in dec] which comes out to be +/- 0.003235 [in Q0.15 format].

Description

This function is used to do the logistic activation function with 16-bit input datatype.

Name: nInputMultiplier

Type: int32_t

Direction: Input

Description: Input param which is the input multiplier.

Name: nInputLeftShift

Type: int32_t

Direction: Input

Description: Input param which is the input left shift value.

Name: nInputSize

Type: int32_t

Direction: Input

Description: Input param which is the input buffer size.

Name: pInputData

Type: const int16_t*

Direction: Input

Description: Input param which is the input buffer size.

Name: pOutputData

Type: const int16_t*

Direction: Output

Description: Input param which is the output buffer size.

Return value.

None

2.2.6.2 adi_sharcfx_logistic_int8

```
void adi_sharcfx_logistic_int8 (
            int32_t nInputZeroPoint,
            int32_t nInputMultiplier,
            int32_t nInputLeftShift,
            int32_t nInputSize,
            const int8_t* pInputData,
            int8_t* pOutputData
);
```

Description

This function is used to do the logistic activation function with 8-bit input datatype.

Name: nInputZeroPoint
Type: int32_t
Direction: Input
Description: Input param which is the input zero point.

Name: nInputMultiplier
Type: int32_t
Direction: Input
Description: Input param which is the input multiplier.

Name: nInputLeftShift
Type: int32_t
Direction: Input
Description: Input param which is the input left shift value.

Name: nInputSize
Type: const int32_t
Direction: Input
Description: Input param which is the input buffer size.

Name: pInputData
Type: const int8_t*
Direction: Input
Description: Input param which is the input buffer size.

Name: pOutputData
Type: int8_t*

Direction: Output

Description: Input param which is the output buffer size.

Return value.

None.

2.2.6.3 adi_sharcfx_tanh_int16

```
void adi_sharcfx_tanh_int16 (
            int32_t nInputMultiplier,
            int32_t nInputLeftShift,
            int32_t nLength,
            const int16_t* pInputData,
            int16_t* pOutputData
);
```

NOTE:

This version of the optimized tanh function has a tolerance of +/- 55[in dec] which comes out to be +/- 0.001678 [in Q0.15 format].

Description

This function is used to do the tanh activation function with 16-bit input datatype.

Name: nInputMultiplier

Type: int32_t

Direction: Input

Description: Input param which is the input multiplier.

Name: nInputLeftShift

Type: int32_t

Direction: Input

Description: Input param which is the input left shift value.

Name: nLength

Type: int32_t

Direction: Input

Description: Input param which is the input length.

Name: pInputData

Type: const int16_t*

Direction: Input

Description: Pointer to the input buffer.

Name: pOutputData

Type: const int16_t*

Direction: Output

Description: Pointer to the output buffer.

Return value.

None

2.2.6.4 adi_sharcfx_relu_int8

```
void adi_sharcfx_relu_int8(
      const int8_t* pInput,
      int8_t* pOutput,
      const uint32_t nSize,
      uint32_t pQuantizedMultiplier,
      int32_t pQuantizedShift,
      int32_t pInOffset,
      int32_t pOutOffset,
      int32_t output_activation_min,
      int32_t output_activation_max);
```

Description

This function applies the ReLU (Rectified Linear Unit) activation function to an int8 input buffer. It performs quantization scaling and clamps the output within a specified activation range.

Parameters

Name: pInput
Type: const int8_t *
Direction: Input
Description: Pointer to the input buffer.

Name: pOutput
Type: int8_t *
Direction: Output
Description: Pointer to the output buffer.

Name: nSize
Type: const uint32_t
Direction: Input
Description: Number of elements in the input buffer.

Name: pQuantizedMultiplier
Type: uint32_t
Direction: Input
Description: Quantized multiplier used for scaling the input.

Name: pQuantizedShift
Type: int32_t
Direction: Input
Description: Quantized shift used for scaling the input.

Name: pInOffset
Type: int32_t
Direction: Input
Description: Input zero-point offset.

Name: pOutOffset

Type: int32_t

Direction: Input

Description: Output zero-point offset.

Name: output_activation_min

Type: int32_t

Direction: Input

Description: Minimum value for activation clamping.

Name: output_activation_max

Type: int32_t

Direction: Input

Description: Maximum value for activation clamping.

Return value.

None

2.2.7 Elementwise Operations

2.2.7.1 adi_sharcfx_elementwise_add_int16

```
void adi_sharcfx_elementwise_add_int16(
      const int16_t* pInput1,
      const int16_t* pInput2,
      int32_t nBatches,
      int32_t nInputLen,
      int16_t* pOutput,
      int32_t kInt16Max,
      int32_t kInt16Min);
```

Description

This function performs element-wise addition of two int16 input arrays across multiple batches. The result is clamped between specified minimum and maximum values.

Parameters

Name: pInput1

Type: const int16_t*

Direction: Input

Description: Pointer to the first input buffer.

Name: pInput2

Type: const int16_t*

Direction: Input

Description: Pointer to the second input buffer.

Name: nBatches

Type: int32_t

Direction: Input

Description: Number of batches to process.

Name: nInputLen

Type: int32_t

Direction: Input

Description: Number of elements in each input buffer per batch.

Name: pOutput

Type: int16_t*

Direction: Output

Description: Pointer to the output buffer.

Name: kInt16Max

Type: int32_t

Direction: Input

Description: Maximum value for activation clamping.

Name: kInt16Min

Type: int32_t

Direction: Input

Description: Minimum value for activation clamping.

Return value.

None

2.2.7.2 adi_sharcfx_elementwise_mul_int16

```
void adi_sharcfx_elementwise_mul_int16(
      const int16_t* pInput1,
      const int16_t* pInput2,
      int16_t* pOutput,
      int32_t nSize,
      uint32_t pQuantizedMultiplier,
      int32_t pQuantizedShift,
      int32_t pInOffset1,
      int32_t pInOffset2,
      int32_t pOutOffset,
      int32_t output_activation_min,
      int32_t output_activation_max)
```

Description

This function performs element-wise multiplication of two int16 input arrays, applies quantization, and clamps the result within a specified activation range, stores output as int16 array.

Parameters

Name: pInput1

Type: const int16_t *

Direction: Input

Description: Pointer to the first input buffer.

Name: pInput2
Type: const int16_t *
Direction: Input
Description: Pointer to the second input buffer.

Name: pOutput
Type: int16_t *
Direction: Output
Description: Pointer to the output buffer.

Name: nSize
Type: int32_t
Direction: Input
Description: Number of elements in the input buffers.

Name: pQuantizedMultiplier
Type: uint32_t
Direction: Input
Description: Quantized multiplier used for scaling the result.

Name: pQuantizedShift
Type: int32_t
Direction: Input
Description: Quantized shift used for scaling the result.

Name: pInOffset1
Type: int32_t

Direction: Input

Description: Zero-point offset for the first input.

Name: pInOffset2

Type: int32_t

Direction: Input

Description: Zero-point offset for the second input.

Name: pOutOffset

Type: int32_t

Direction: Input

Description: Zero-point offset for the output.

Name: output_activation_min

Type: int32_t

Direction: Input

Description: Minimum value for activation clamping.

Name: output_activation_max

Type: int32_t

Direction: Input

Description: Maximum value for activation clamping.

Return value.

None

2.2.7.3 adi_sharcfx_elementwise_mul_int8

```
void adi_sharcfx_elementwise_mul_int8(
      const int16_t* pInput1,
      const int16_t* pInput2,
      int8_t* pOutput,
      int32_t nInputLen,
      uint32_t nQuantizedMultiplier,
      int32_t nQuantizedShift,
      int32_t nInOffset1,
      int32_t nInOffset2,
      int32_t nOutOffset,
      int32_t output_activation_min,
      int32_t output_activation_max);
```

Description

This function performs element-wise multiplication of two int16 input arrays, applies quantization, and clamps the result within a specified activation range, stores output as int8 array.

Parameters

Name: pInput1

Type: const int16_t*

Direction: Input

Description: Pointer to the first input buffer.

Name: pInput2

Type: const int16_t*

Direction: Input

Description: Pointer to the second input buffer.

Name: pOutput

Type: int8_t*

Direction: Output

Description: Pointer to the output buffer.

Name: nInputLen
Type: int32_t
Direction: Input
Description: Input param which is the number of elements in the input buffers.

Name: nQuantizedMultiplier
Type: uint32_t
Direction: Input
Description: Input param which is the quantized multiplier used for scaling the result.

Name: nQuantizedShift
Type: int32_t
Direction: Input
Description: Input param which is the quantized shift used for scaling the result.

Name: nInOffset1
Type: int32_t
Direction: Input
Description: Input param which is the zero-point offset for the first input.

Name: nInOffset2
Type: int32_t
Direction: Input
Description: Input param which is the zero-point offset for the second input.

Name: nOutOffset
Type: int32_t

Direction: Input

Description: Input param which is the zero-point offset for the output.

Name: output_activation_min

Type: int32_t

Direction: Input

Description: Input param which is the minimum value for activation clamping.

Name: output_activation_max

Type: int32_t

Direction: Input

Description: Input param which is the maximum value for activation clamping.

Return value.

None

2.2.8 LSTM

Description

LSTMs in TFLM are defined internally as a combination of kernels. All the sub-kernels that LSTM calls have been optimized [Fully Connected layer, Tanh and Sigmoid/Logistic and elementwise ops] and substituted in the main LSTM kernel.

NOTE:

Empirically we have noticed an error in the order of 1e-2 in LSTM kernels when used with optimized implementations of tanh and sigmoid activations, arising from the limited resolution of the approximation used in both these activations.

3 Appendix A-Example Code

```
void adi_sharcfx_logistic_int16 (
      int32_t nInputMultiplier,
      int32_t nInputLeftShift,
      int32_t nInputSize,
      const int16_t* pInputData,
      int16_t* pOutputData)
{
int16_t* pInput_in_q3_12 = (int16_t*)nQFormatBuffer;
xb_vec2Mx16 *inp = (xb_vec2Mx16 *)pInputData;
xb_vec2Mx16 *outp;
outp=(xb_vec2Mx16 *)pInput_in_q3_12;
xb_vec2Mx16 vin;
valign ina,outa;
ina=PDX_LA_2MX16_PP (inp);
outa = PDX_Z_ALIGN();
if (nInputMultiplier == 0)
{
      pInput_in_q3_12 = (int16_t*)pInputData;
}
else
{
      xb_vec2Mx40 vTempOut;
      int32_t nTempRound = nInputLeftShift > 0 ? (1<<(nInputLeftShift-1)) : 0;
      for (int i = 0; i < nInputSize; i += 2*PDX_M)
      {
            PDX_LA_2MX16_XP (vin, ina, inp, (2*PDX_M)*2);
            vTempOut = PDX_MULW_2MX16(vin,nInputMultiplier);
            vTempOut = PDX_ADD_2MX40(vTempOut,nTempRound);
            vTempOut = PDX_SRA_2MX40(vTempOut,nInputLeftShift);
            vTempOut = PDX_PACKSIV_2MX40(vTempOut,0)
            PDX_SAV_2MX16_XP(vTempOut,outa,outp, (2*PDX_M)*2);
            PDX_SAPOS_2MX16_FP( outa, outp );
      }
}
vecsigmoid_16b(pInput_in_q3_12, pOutputData , nInputSize);
}
```